

Using Methods

More on writing methods

Produced Dr. Siobhán Drohan
by: Mr. Colm Dunphy
 Ms Mairead Meagher
 Ms Siobhan Roche

Topics list



1. RECAP: Method Terminology:
2. Method examples – return values
 - Celcius / Farenheit **Converter**.
 - **Cubed**
 - **Product**
3. **Overloading** methods.
4. **Recursion**.

Simple Methods

- A method is a named block of code, that can be invoked later

```
Public void printHello () {  
    System.out.println("Hello!");  
}
```

Method terminology

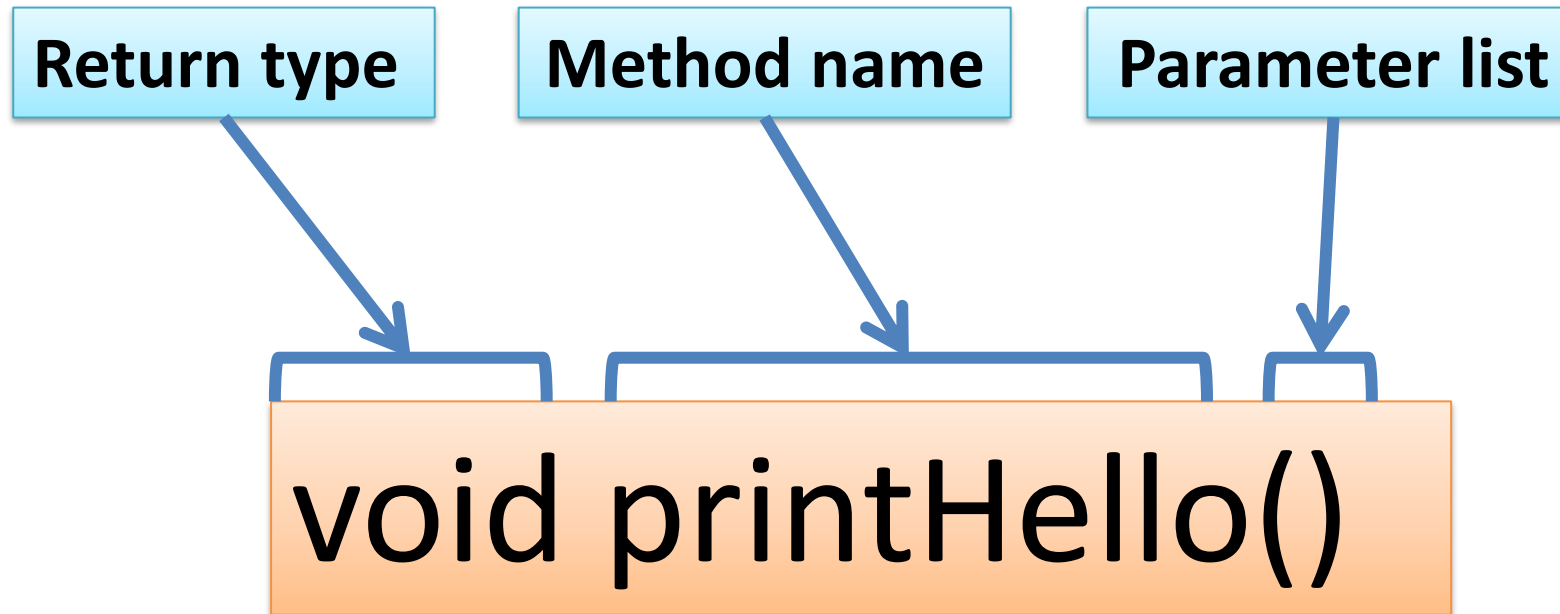
Method **signature** / header

Method **body**

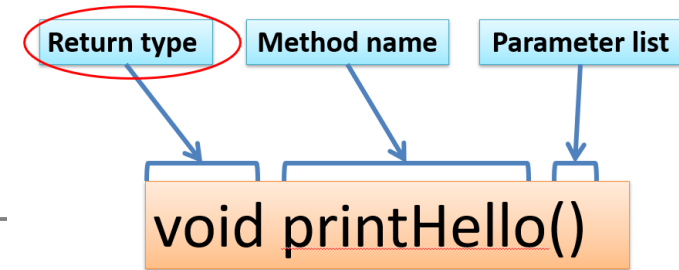
```
void printHello()  
{  
    System.out.println("Hello");  
}
```

The diagram illustrates the components of a Java method. A light blue box labeled 'Method signature / header' has an arrow pointing to the first line of the code example, 'void printHello()'. Another light blue box labeled 'Method body' has an arrow pointing to the opening curly brace of the code example. The code example itself is contained within an orange box and shows a method named 'printHello' that returns 'void' and prints 'Hello' to the console.

Method signature



Return Type

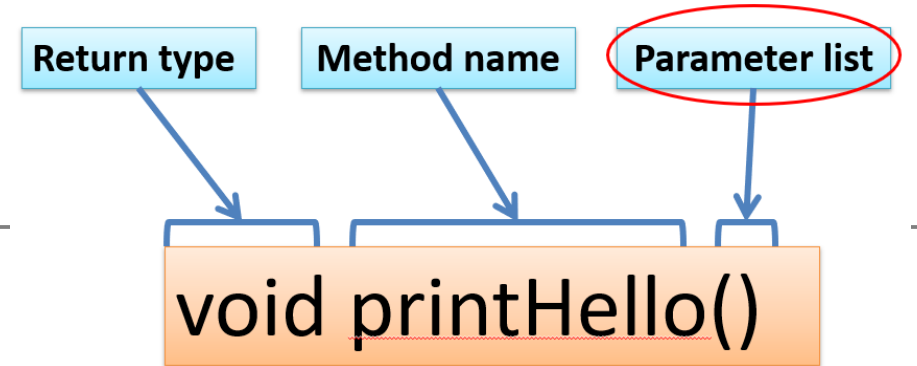


- Methods can return information.
- The **void** keyword just before the method name means that **nothing is returned** from the method.
- When a data type (e.g. **int**) appears before the method name, this means that **something is returned** from the method.
- Within the body of the method, you use the **return** statement to **return the value**.

```
int timesTwo(int number)  
{  
    number = number * 2;  
    return number;  
}
```

Parameter list

- Methods **take in data** via their **parameters**.
- Methods do not have to pass parameters
- A **parameter** is a **variable declaration** –
 - it has a **type** (e.g. int) and a **name** (e.g. num).
- If a method needs additional information to execute, we provide a parameter, so that the information can be passed into it.
- Methods can have **any number of parameters**



```
int timesTwo(int number)
{
    number = number * 2;
    return number;
}
```

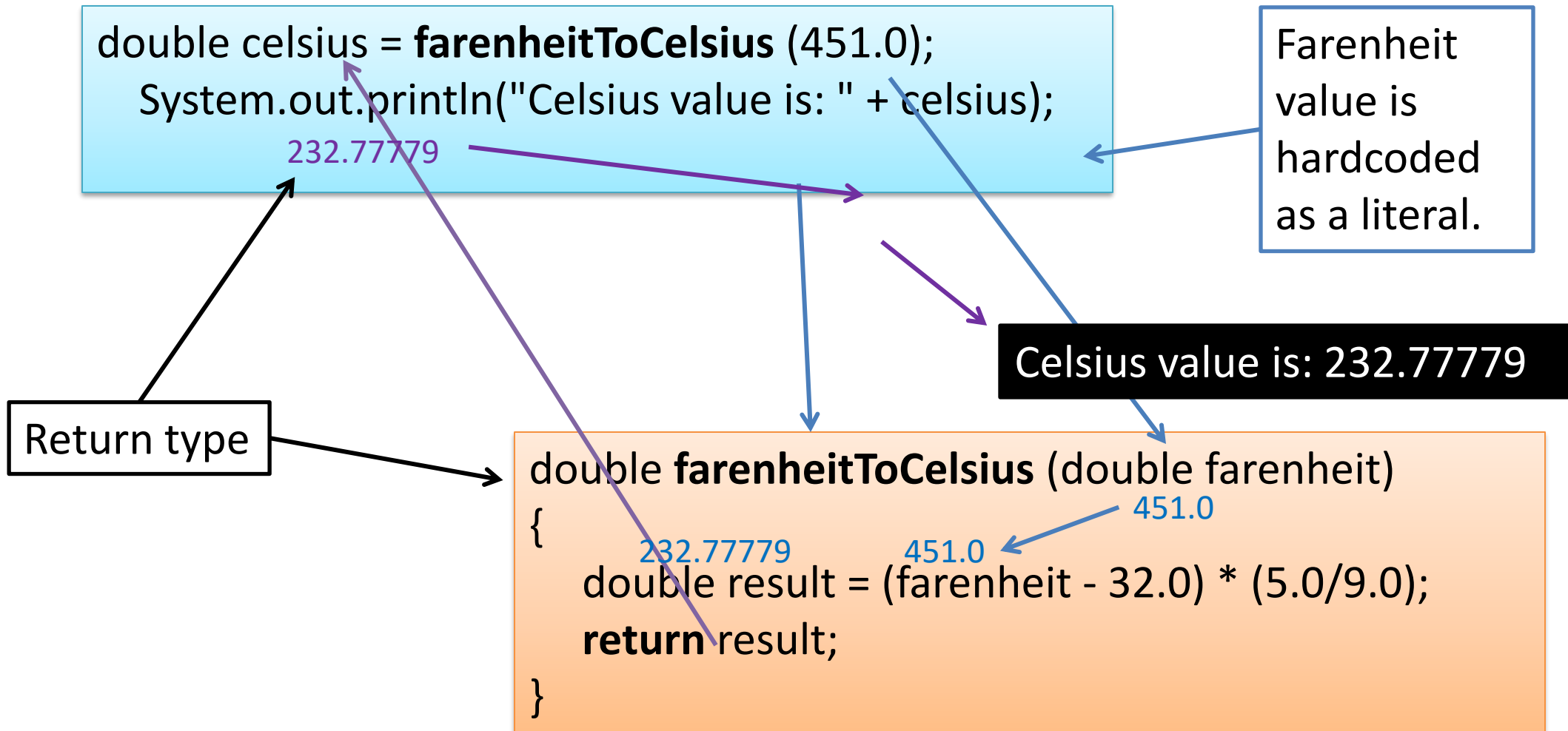
Topics list

1. RECAP: Method Terminology:
2. Method examples – return values
 - Celcius / Farenheit **Converter**.
 - **Cubed**
 - **Product**
3. **Overloading** methods.
4. **Recursion**.

Return a double

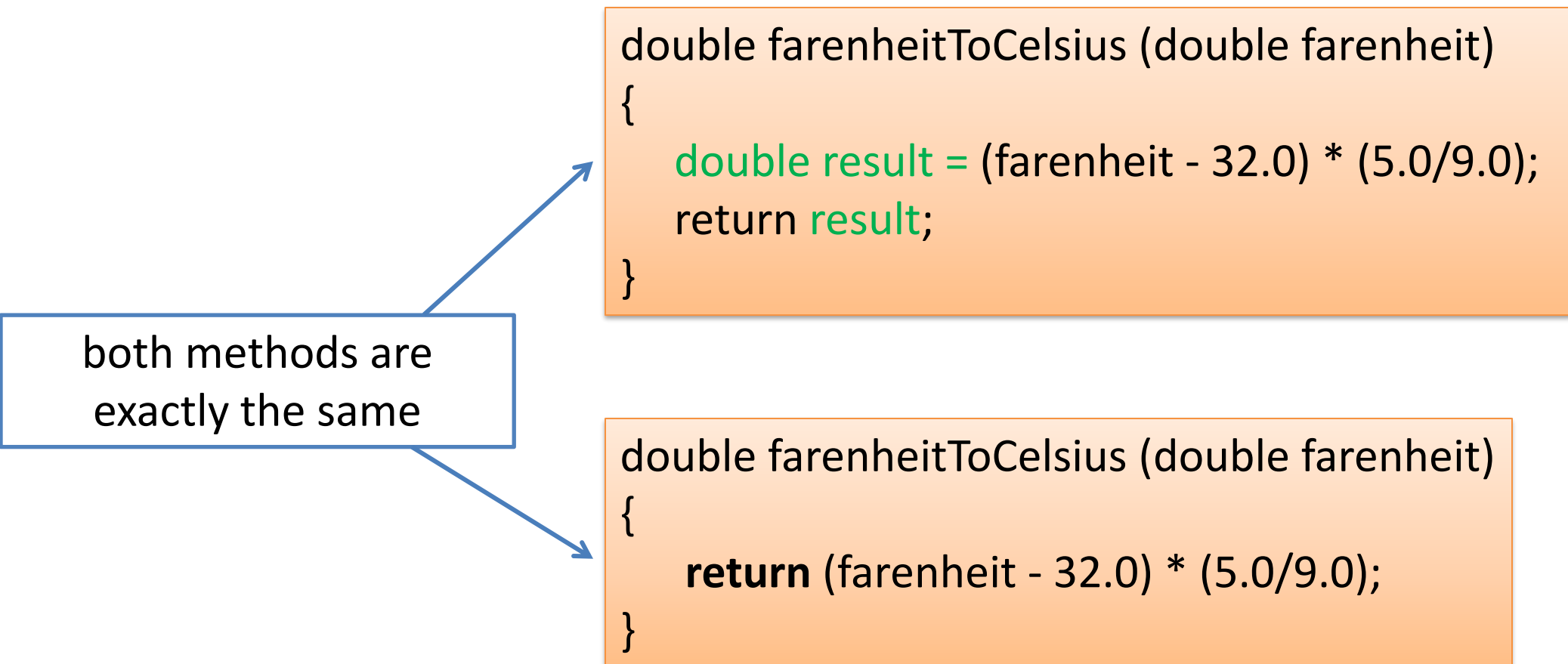
- Now we consider an example where the method **does return a value** (which could be any data type) but in this next example it returns a **double**

Example – Fahrenheit to Celsius



Example – Updated

both methods are
exactly the same



```
double fahrenheitToCelsius (double fahrenheit)
{
    double result = (fahrenheit - 32.0) * (5.0/9.0);
    return result;
}
```

```
double fahrenheitToCelsius (double fahrenheit)
{
    return (fahrenheit - 32.0) * (5.0/9.0);
}
```

Topics list

1. RECAP: Method Terminology:
2. Method examples – return values
 - Celcius / Farenheit **Converter**.
 - **Cubed**
 - **Product**
3. **Overloading** methods.
4. **Recursion**.

Example: Cubed

© ClassExamples.java x

```
1 public class ClassExamples { 2
2
3 int cubed(int num){ ← 1 usage
4     return num * num * num;
5 }
6
7
```

Driver.java x

```
1 public class Driver {
2
3 public static void main(String[] args) {
4     ClassExamples ce = new ClassExamples();
5     int numToBeCubed = 5;
6     int cubedAns = ce.cubed(numToBeCubed);
7     System.out.println(numToBeCubed + " cubed is " + cubedAns);
8 }
9
10
```

Annotations in the code:

- A blue arrow points from the `5` in `numToBeCubed = 5;` to the `int cubed(int num)` method signature in `ClassExamples.java`.
- A green arrow points from the `125` in `int cubedAns = ce.cubed(numToBeCubed);` to the `return num * num * num;` line in `ClassExamples.java`.
- A black arrow points from the `Return type` box to the `int` in `int cubed(int num)`.

Return type

Topics list

1. RECAP: Method Terminology:
2. Method examples – return values
 - Celcius / Farenheit **Converter**.
 - **Cubed**
 - **Product**
3. **Overloading** methods.
4. **Recursion**.

Example: Product

- Driver.java x ClassExamples.java

```
1  public class Driver {  
2  
3  public static void main(String[] args) {  
4      ClassExamples ce = new ClassExamples();  
5      double a = 20.5;  
6      double b = 3.5;  
7      System.out.println(" The product of " + a + " and " + b + " is " + ce.product(a, b));  
8  }  
9  }  
10
```

```
Driver.java  ClassExamples.java x  
public class ClassExamples { 2 usages  
    double product(double num1, double num2){  
        return num1 * num2;  
    }  
}
```

Return type

The product of 20.5 and 3.5 is 71.75

Questions?

